KCPC Week 5 Problems
Graphs II: Dijkstra and 0–1 BFS

This sheet accompanies the *Graphs II* workshop, hosted in collaboration with KCL Tech. Problems are divided into three categories, numbered continuously 1–15 to match the solutions document:

- **Integrated examples (1–3)**: worked through live in the slides, with try-it-yourself checkpoints.
- **Core practice (4–11)**: practised independently. The slides' try-it-yourself checkpoints cover the same ideas, and the Solutions give full answers.
- **Extended homework (12–15)**: stretch challenges for after the session.

How to approach every problem. Before coding, write down the four modelling decisions from the slides:

1. **State**: what is a vertex (a city, a grid cell, a city–time pair, ...)?
2. **Legal moves**: what are the edges, and are they directed?
3. **Move cost**: uniform, binary $\{0, 1\}$, or general non-negative?
4. **Initial score**: single source or many; is the starting cell itself charged?

Then choose the algorithm by the model, not by the problem's title: uniform positive cost $\Rightarrow$ BFS; costs in $\{0, 1\}$ $\Rightarrow$ 0–1 BFS; general non-negative costs $\Rightarrow$ Dijkstra. For maximisation or bottleneck objectives, adapt the relaxation and the heap ordering, and justify why a current best pop is final.

I. Integrated examples (live walkthroughs)

1. GeeksForGeeks — Dijkstra Algorithm

Modelling cue: undirected non-negative weights; return every distance from the source.

Note. The historical title says *Adjacency Matrix*, but the current judge interface is an **edge list**: `dijkstra(V, edges, src)` with rows `[u, v, w]`. Build both adjacency directions.

2. LeetCode 743 — Network Delay Time

Modelling cue: directed edges; a vertex's receive time is its shortest distance; answer is the maximum distance, or -1 if any vertex is unreachable. Labels are 1-based.

3. LeetCode 1368 — Minimum Cost to Make at Least One Valid Path in a Grid

Modelling cue: leaving a cell costs 0 along its arrow and 1 otherwise; the arrow belongs to the cell you *leave*. Use 0–1 BFS.

II. Core practice

4. LeetCode 1514 — Path with Maximum Probability

Modelling cue: maximise a *product* of probabilities, not a sum. Start with best$[s] = 1$ and extend by multiplying; use a max-heap (negate keys in Python). A current maximum pop is final, because extending a route cannot increase its probability.

5. LeetCode 1631 — Path with Minimum Effort

Modelling cue: minimise the *largest* height difference on the path. Start at effort 0; the score is a bottleneck, not a sum. Extend with $\max(\text{effort}[u], |h[u] - h[v]|)$; keep a min-heap.

6. LeetCode 2290 — Minimum Obstacle Removal to Reach Corner

Modelling cue: the cost belongs to the *destination* cell: entering an obstacle costs 1. Use 0–1 BFS. Both endpoints are empty. Do not charge the cell you leave; contrast with LC1368 (Problem 3).

7. LeetCode 542 — 01 Matrix

Modelling cue: multi-source *ordinary* BFS: enqueue every zero at distance 0; first discovery in FIFO order is minimal. One BFS covers all zeros, in $O(RC)$ time. Not a 0–1 BFS problem.

8. AtCoder ABC176 D — Wizard in Maze

Modelling cue: walking to an open orthogonal neighbour costs 0; warping to any open cell with $|\Delta r| \le 2$ and $|\Delta c| \le 2$ (including corners) costs 1. Use 0–1 BFS. Input coordinates are 1-based; output -1 if unreachable.

9. Project Euler 81 — Path Sum: Two Ways

Modelling cue: moves are right and down only; entering a cell costs its value, including the starting cell. A row-by-row DP also works on this DAG.

10. Project Euler 82 — Path Sum: Three Ways

Modelling cue: moves are up, down and right; *any* left-column cell may start (seed each at its own value) and *any* right-column cell may finish.

11. Project Euler 83 — Path Sum: Four Ways

Modelling cue: all four orthogonal moves; top left to bottom right; destination-entry costs. Use Dijkstra (or 0–1 reasoning does *not* apply: values are general positives).

III. Extended homework

Important. The following problems are genuinely extremely difficult. Attempt them only after completing all previously listed problems and the assigned supplementary reading. They are stretch challenges for exploring advanced modelling and optimisation techniques. Solutions are provided in C++.

12. LeetCode 1928 — Minimum Cost to Reach Destination in Time

Modelling cue: state expansion: the cheapest arrival at a city may be too slow to finish. Keep (city, timeUsed) in the state; the source fee and each entered city's fee are charged.

13. Codeforces 786B — Legacy

Modelling cue: segment-tree graph construction with *two directed trees* (out-tree and in-tree sharing original leaves) plus Dijkstra. Do not expand ranges into individual edges.

14. Codeforces 715B — Complete The Graph

Modelling cue: constructive weights: activate missing edges one at a time (weight 1 vs. disabled at $L + 1$), binary-search the pivot whose activation crosses the target distance, then adjust only the pivot. Vertices are 0-based; input weight 0 means *unknown*.

15. AtCoder ABC213 E — Stronger Takahashi

Modelling cue: advanced 0–1 movement modelling: one punch clears a 2×2 square. Free edges enter only originally open orthogonal neighbours; paid edges use every nonzero offset in the 5×5 square *except* the four corners $(\pm 2, \pm 2)$, and may land on walls.